

# EduGenie: AI-Powered Learning Assistant

Team ID: SWTID-2026-7019 | Date: 29 September 2026

## Project Description:

EduGenie harnesses Generative AI to provide an intelligent, personalized learning-support experience for students. The platform includes a Q&A module that answers any academic question, an Explanation module that breaks concepts down in simple, school-friendly language, a Summarizer that condenses long passages, a Quiz Generator that produces three multiple-choice questions from a topic or passage, and a Learning Path module that returns a structured beginner-to-advanced roadmap for any subject.

Utilizing the Google Gemini API through the google-genai SDK, together with a locally hosted LaMini-Flan-T5-783M model for offline explanations, EduGenie processes user inputs and delivers fast, clear educational content. Built with FastAPI and served with Uvicorn, the platform runs locally with a clean single-page interface. With secure API key management via .env and structured error handling on every endpoint, EduGenie empowers students, teachers, and self-learners to study with confidence.

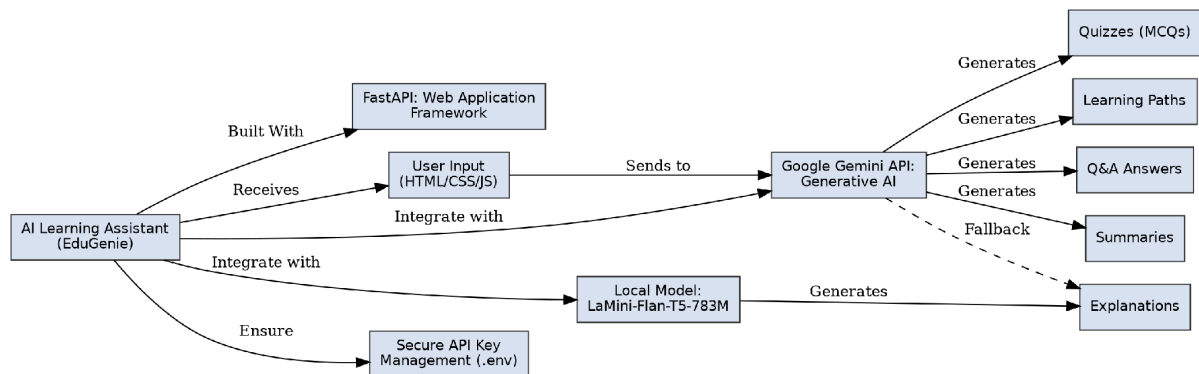
## Scenarios

**Scenario 1:** A student types “Why is the sky blue?” into the Q&A card. The system sends the question to Gemini and returns a clear, accurate answer in seconds, right below the input box.

**Scenario 2:** A teacher pastes a long passage on the water cycle into the Quiz card. EduGenie generates three multiple-choice questions with four options each. The student clicks an option, sees instant green or red feedback, and receives a final score with an encouraging message.

**Scenario 3:** A self-learner enters “SQL” in the Learning Path card. The AI tutor returns a structured roadmap covering beginner, intermediate, and advanced topics, the order in which to learn them, and suggested videos, articles, and books.

## Technical Architecture:



**Description:** EduGenie utilizes a modular three-tier architecture to deliver rapid, AI-driven learning support. The frontend provides a responsive user interface built with HTML, CSS, and vanilla JavaScript, while the FastAPI backend orchestrates request validation and prompt construction. Each feature lives in its own Python module (qna.py, explanation\_module.py, summary\_module.py, quiz\_module.py, learning\_path.py) that interfaces with the Google Gemini API; the Explanation module additionally loads a local Hugging Face model (MBZUAI/LaMini-Flan-T5-783M) and falls back to Gemini if the model cannot be loaded. The application is served locally with Uvicorn.

(Note: This project does not use a database. All content is generated on demand and nothing is stored, so no ER diagram is required.)

## Pre-requisites:

- **FastAPI Framework Knowledge:** [FastAPI Documentation](#)
- **Google Gemini API Familiarity:** [Gemini API Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Hugging Face Transformers Basics:** [Transformers Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Uvicorn Documentation](#)

# Project Workflow:

## Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI models (Gemini for cloud tasks and LaMini-Flan-T5 for local explanations).
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for FastAPI, Gemini, and Transformers.

## Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities
- **Activity 2.2:** Implement the FastAPI backend to manage routing and user input processing, ensuring smooth API interactions.

## Milestone 3: main.py Development

- **Activity 3.1:** Write the main application logic in main.py, establishing routes for each feature and integrating AI responses.

## Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic rendering with the Fetch API to display results based on user interactions.

## Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Run the application locally with Uvicorn and verify every feature end to end.

## Milestone 6: Conclusion

## Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI models for our learning-assistant needs. This involves researching the capabilities and performance of the models available, ensuring that the chosen models align well with the application's objectives of answering questions, explaining concepts, summarizing text, generating quizzes, and recommending learning paths.

### Activity 1.1: Generate a Gemini API Key

Before the application can communicate with Gemini, you need a valid API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Open Google AI Studio:** Visit <https://aistudio.google.com> and sign in with your Google account.

*[ Insert screenshot: Google AI Studio home page ]*

- **Step 2 — Navigate to API Keys:** Open <https://aistudio.google.com/app/apikey>, or choose "Get API key" from the left-hand menu.
- **Step 3 — Create a New API Key:** Click "Create API key", select a Google Cloud project (or create a new one), and give the key a descriptive name (e.g., edugenie-key).

*[ Insert screenshot: Create API key dialog ]*

- **Step 4 — Copy the Key:** Copy the generated key immediately and store it in a safe place.
- **Step 5 — Add the Key to Your Project:** Copy `.env.example` to a new file named `.env` in the project root, and paste the key as shown below:

```
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_MODEL=gemini-3-flash-preview
LOCAL_EXPLANATION_MODEL=MBZUAI/LaMini-Flan-T5-783M
```

- **Step 6 — Verify the Key is Loaded:** `main.py` loads the key automatically at startup using `python-dotenv` and rejects requests with a clear error if the key is missing:

```
from dotenv import load_dotenv
load_dotenv()
```

```
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "")  
GEMINI_MODEL   = os.getenv("GEMINI_MODEL", "gemini-2.0-flash")
```

- **Important — Never Commit Your API Key:** Keep .env listed in .gitignore, and keep only placeholder values in .env.example:

```
.env
```

## Activity 1.2: Research and Select the Appropriate Generative AI Models

Here are the things that needed to be researched to select the best models for the project:

- **Understand the Project Requirements:** Review the specific needs of EduGenie, focusing on the types of content it will generate (answers, explanations, summaries, quizzes, and learning paths for students).
- **Explore the Model Documentation:** Visit the Gemini API documentation and the Hugging Face model hub to examine the available models, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to educational tasks.
- **Select the Optimal Models:** Choose a Gemini Flash model (configurable through GEMINI\_MODEL, e.g., gemini-3-flash-preview) for its balance of answer quality and speed across Q&A, summarization, quiz generation, and learning paths. Choose MBZUAI/LaMini-Flan-T5-783M as a lightweight local model for simple explanations (max\_new\_tokens 150, temperature 0.7, top\_k 50, top\_p 0.95), with automatic fallback to Gemini if it cannot be loaded.

## Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components like the frontend (HTML, CSS, JavaScript), FastAPI backend, Gemini API integration, and the local explanation model.
- **Detail Frontend Functionality:** Outline how users interact with the application: typing a question, topic, or passage into one of five feature cards and clicking the action button to see the result below it.
- **Outline Backend Responsibilities:** Specify how the FastAPI backend receives GET and POST requests, validates that the required input is present, checks that the API key is configured, calls the matching module, and returns JSON.
- **Describe AI Integration Points:** Define how each module builds its prompt, calls `genai.Client(...).models.generate_content()`, cleans the response (for example, `clean_json_block()` strips markdown fences from quiz output before `json.loads()`), and returns structured results to the frontend.

## Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.10+ is installed on your machine along with pip for dependency management (the project was developed on Python 3.13).

- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects\EduGenie-Learning-assistant>python -m venv venv
D:\projects\EduGenie-Learning-assistant>venv\Scripts\activate
(venv) D:\projects\EduGenie-Learning-assistant>pip install -r requirements.txt
```

- **Install Dependencies:** requirements.txt installs FastAPI, Uvicorn, Jinja2, python-multipart, google-genai, transformers, torch, sentencepiece, and python-dotenv.

```
fastapi
uvicorn
jinja2
python-multipart
google-genai
transformers
torch
sentencepiece
python-dotenv
```

- **Configure the API Key:** Create a .env file in the project root and add your Gemini API key (see Activity 1.1).
- **Set Up Application Structure:** Create the project directory structure including folders for templates and static files, and the main application files (main.py and the five feature modules).

```
EduGenie-Learning-assistant/
├── static/
│   └── style.css
├── templates/
│   └── index.html
├── .env
├── .env.example
├── .gitignore
├── main.py
├── qna.py
├── explanation_module.py
├── summary_module.py
├── quiz_module.py
├── learning_path.py
└── requirements.txt
```

## Milestone 2: Core Functionalities Development

### Activity 2.1: Develop Core Learning Features

Implement the five core features as independent modules, each wrapping a single, focused prompt.

- **Q&A (qna.py):** Sends the student's question straight to Gemini and returns the answer text.
- **Explanation (explanation\_module.py):** Explains a concept "in a simple and clear way for a school student", using the local LaMini-Flan-T5 model and falling back to Gemini when the local model is unavailable.
- **Summarizer (summary\_module.py):** Summarizes any pasted text in simple language.
- **Quiz Generator (quiz\_module.py):** Creates three multiple-choice questions, each with four options and a correct answer that exactly matches one option, returned as valid JSON.
- **Learning Path (learning\_path.py):** Acts as an AI tutor and returns a structured path with key topics, order of learning, and resources (videos, articles, books) across beginner, intermediate, and advanced levels.

### Activity 2.2: Implement the FastAPI Backend

#### Define Routes in FastAPI:

- Set up routes in main.py for the home page and for each feature endpoint.

#### Process User Input:

- Create five input cards in index.html for users to submit their question, topic, or text.
- Use `await request.json()` to retrieve JSON data submitted from the frontend via the Fetch API (POST endpoints) and `Query(...)` parameters for GET endpoints.
- Validate that the required field (topic, text) is present before processing, returning HTTP 400 otherwise.

#### Integrate API Calls:

- Within each route handler, first call `_check_api_key()` to confirm the Gemini key is configured.
- Call the matching module function (for example, `generate_quiz()`) which submits the prompt to Gemini using `client.models.generate_content()`.
- Return the result as JSON; modules catch exceptions and return a readable [ERROR] message so the UI never breaks.

## Milestone 3: main.py Development

Milestone 3 focuses on creating and refining the core logic of the main.py file, which serves as the backbone of the EduGenie application. In this milestone, you'll set up each feature's functionality through FastAPI routes, add a global JSON error handler, configure CORS, and connect every route to its AI module.

### Activity 3.1: Writing the Main Application Logic in main.py

#### Create the App, Middleware, Static Files and Templates:

```
app = FastAPI(title="EduGenie - AI Learning Assistant")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5500", "http://127.0.0.1:5500",
                  "http://localhost:3000", "http://127.0.0.1:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)

app.mount("/static", StaticFiles(directory="static"), name="static")
templates = Jinja2Templates(directory="templates")
```

#### Add a Global Exception Handler and API-Key Check:

```
@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    return JSONResponse(status_code=500,
                        content={"error": f"Server error: {str(exc)}"})

def _check_api_key():
    if not GEMINI_API_KEY or GEMINI_API_KEY == "your_gemini_api_key_here":
        return JSONResponse(status_code=500, content={
            "error": "GEMINI_API_KEY is not configured. ..."})
    return None
```

#### Define the Core Routes in main.py:

Establish routes for EduGenie's six endpoints, each serving a distinct purpose:

- **/ (GET)** — Home page route that renders the main index.html interface.

```
@app.get("/")
async def index(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})
```

- **/qa (GET)** — accepts a question query parameter and returns {"answer": ...}.

```
@app.get("/qa")
async def answer_question(question: str = Query(...)):
    if (err := _check_api_key()): return err
    answer = answer_question_with_gemini(question, GEMINI_API_KEY, GEMINI_MODEL)
    return {"answer": answer}
```

- **/explain/ (POST)** — accepts {"topic": ...} and returns {"topic", "explanation"}.

```
@app.post("/explain/")
async def explain_api(request: Request):
    if (err := _check_api_key()): return err
    data = await request.json()
```

```
topic = data.get("topic")
if not topic:
    return JsonResponse(content={"error": "Please provide a topic."}, status_code=400)
explanation = explain_topic(topic, GEMINI_API_KEY, GEMINI_MODEL)
return {"topic": topic, "explanation": explanation}
```

- **/summarize/ (POST)** — accepts {"text": ...} and returns {"summary": ...}.
- **/quiz (POST)** — accepts {"text": ...} and returns {"quiz": [ {question, options, answer}, ... ]}.

```
@app.post("/quiz")
async def quiz_api(request: Request):
    if (err := _check_api_key()): return err
    data = await request.json()
    text = data.get("text")
    if not text:
        return JsonResponse(content={"error": "Please provide text for quiz."},
status_code=400)
    quiz = generate_quiz(text, GEMINI_API_KEY, GEMINI_MODEL)
    return JsonResponse(content={"quiz": quiz})
```

- **/learn/recommendations (GET)** — accepts a topic query parameter and returns {"topic", "recommendation"}.

### Define Prompt Engineering Helpers:

- **generate\_quiz()**: instructs Gemini to act as a quiz generator, create 3 MCQs with 4 options each, and reply as strict JSON.
- **get\_learning\_recommendations()**: instructs Gemini to act as an AI tutor and suggest a structured, adaptive learning path with resources.

### Integrate the AI Responses in Each Function:

- **clean\_json\_block()**: a small parser that strips markdown code fences from the model's reply before json.loads(), ensuring the quiz JSON parses reliably.

```
def clean_json_block(text):
    # Remove Markdown ```json code fences
    return re.sub(r"```(?:json)?\n(?:.*)```", r"\1", text, flags=re.DOTALL).strip()
```

- Every module wraps its Gemini call in try/except and returns an [ERROR] message instead of raising, so the frontend can always display something meaningful.

## Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and clean interface for EduGenie. This involves designing a card-based layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display AI-generated answers, explanations, summaries, quizzes, and learning paths based on user interactions.

### Activity 4.1: Designing and Developing the User Interface

#### Set Up the Base HTML Structure:

- Develop the main HTML file (templates/index.html) as the single-page interface, including a header with the EduGenie brand and tagline, and five feature cards.
- Use semantic HTML elements (header, main, section) to keep the structure organized and accessible.
- Use Google Fonts (Inter) for clean typography.

#### Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/style.css) implementing a clean, light card-based design with CSS variables for colors.
- Use flexbox layouts for the input rows (text input + action button) and stacked columns for textarea inputs.
- Add a media query (max-width: 540px) so input rows stack vertically and buttons expand full width on mobile screens.

#### Create Separate UI Components for Each Output Type:

- **Result Boxes:** answers, explanations, summaries and learning paths render inside a styled result box beneath each card.
- **Quiz Cards:** each question is a card with four option buttons; the correct option turns green and a wrong pick turns red, with a running score bar.
- **Toast Notifications:** brief error or success messages appear when the user submits empty input.

### Activity 4.2: Creating Dynamic Content Rendering with JavaScript

#### Handle Requests Asynchronously:

- Each feature has an async function (askQuestion, explainTopic, summarizeText, generateQuiz, getLearningPath) that reads the input, validates it, and calls the backend with the Fetch API.
- During loading, setLoading() disables the button and changes its label to "Loading..."

#### Render Results Dynamically:

- showResult() injects the response into the result box and smooth-scrolls it into view using scrollIntoView().
- renderQuiz() builds the question cards, and checkAnswer() locks each question after one attempt, highlights the correct answer and shows the final score with a message ("Perfect!", "Good job!" or "Keep practicing!").

**Restore UI State After Generation:**

- In the finally block of each async handler, the button is re-enabled and its original label restored regardless of success or failure.

## Milestone 5: Deployment

In Milestone 5, the focus is on deploying the EduGenie application locally to verify correct operation. This involves configuring the server environment, managing dependencies, and launching the app with Uvicorn.

### Activity 5.1: Preparing the Application for Local Deployment

#### Set Up a Virtual Environment:

- Open a new terminal in the project folder (for example, in VS Code choose Terminal → New Terminal, then select “Command Prompt”).
- Create and activate a virtual environment, then install all required dependencies from requirements.txt.

```
D:\projects\EduGenie-Learning-assistant>python -m venv venv
D:\projects\EduGenie-Learning-assistant>venv\Scripts\activate
(venv) D:\projects\EduGenie-Learning-assistant>pip install -r requirements.txt
```

#### Configure Environment Variables:

- Create a .env file in the project root and add your Gemini API key. The app loads this automatically using python-dotenv at startup:

```
GEMINI_API_KEY=your_gemini_api_key_here
```

### Activity 5.2: Local Testing and Verification

#### Start the FastAPI Server:

- Run the application locally using Uvicorn:

```
(venv) D:\projects\EduGenie-Learning-assistant>uvicorn main:app --reload
```

- Once running, open a browser and navigate to “http://127.0.0.1:8000” to access the app. Interactive API docs are also available at <http://127.0.0.1:8000/docs>.

*[ Insert screenshot: terminal showing Uvicorn running on http://127.0.0.1:8000 ]*

- Interact with every card (Q&A, Explanation, Summarizer, Quiz, Learning Path) to verify the AI responses are correctly generated, parsed, and rendered.

#### Important Notes:

- The first launch downloads the LaMini-Flan-T5-783M model from Hugging Face (about 3 GB with PyTorch), so start-up can take a few minutes and needs internet access.

- If the local model cannot be loaded, the Explanation feature automatically falls back to Gemini.
- The app is configured for local use; it is not exposed through a public URL.

# Exploring the Website's Web Pages:

## Home / Learning Assistant Page:

  
**Welcome to EduGenie**  
Your personal AI tutor for learning support!

---

  
**Ask EduGenie a Question**  
Ask anything — science, history, math, coding...

  
**Need an Explanation?**  
Get a simple, school-friendly explanation of any concept.

  
**Summarize a Paragraph**  
Paste long content and get a concise, clear summary.

  
**Generate a Quiz**  
Enter a topic or paste a passage — get 3 MCQs instantly.

  
**Get a Learning Path**  
Enter a topic and get a structured beginner-to-advanced roadmap.

**Description:** The single-page interface of EduGenie serves as both the landing page and the learning tool. It features a clean card-based design, a header displaying the EduGenie brand (brain + sparkles logo) and the tagline “Your personal AI tutor for learning support!”, followed by five feature cards stacked vertically: Ask a Question, Need an Explanation?, Summarize a Paragraph, Generate a Quiz, and Get a Learning Path.

## Q&A and Explanation Section:

[ Insert screenshot: Q&A and Explanation cards showing generated answers ]

**Description:** The student types a question or a topic into a text field and clicks Get Answer or Explain. While the AI processes the request the button shows “Loading...”, and the result appears in a result box directly under the card.

### Summarizer Section:

*[ Insert screenshot: Summarize a Paragraph card showing a generated summary ]*

**Description:** A textarea accepts long pasted content. After clicking Summarize, a concise summary in simple language is displayed in the result box.

### Quiz Section:

*[ Insert screenshot: Generate a Quiz card showing three MCQs with answer feedback and score ]*

**Description:** Revealed dynamically after generation, this section shows three multiple-choice question cards with four options each. Clicking an option immediately highlights the correct answer in green (and a wrong pick in red), and once all questions are answered the score bar displays the final result with an encouraging message.

### Learning Path Section:

*[ Insert screenshot: Get a Learning Path card showing a generated roadmap ]*

**Description:** The student enters a subject such as SQL or Machine Learning and receives a structured roadmap in the result box, covering beginner to advanced topics, the recommended order of learning, and suggested videos, articles, and books.

## Conclusion:

EduGenie is a generative AI learning platform built with FastAPI and styled with a clean card-based UI that streamlines student study support. It uses the Google Gemini API together with a local LaMini-Flan-T5 model to instantly answer questions, explain concepts, summarize passages, generate quizzes, and recommend learning paths. The project, which included model selection, module development, frontend design, and local deployment with Uvicorn, highlights how targeted AI and a modular framework can boost learning productivity for students, teachers, and self-learners, with potential for future scalability. Looking ahead, EduGenie offers significant opportunities for expansion, such as adding user authentication to save study history and quiz scores, supporting multiple languages, allowing document or PDF uploads for summarization and quiz creation, and hosting the platform publicly. By continuously refining the underlying prompts and enhancing the frontend accessibility, the platform is well-positioned to evolve from a specialized study helper into a comprehensive personalized learning assistant for learners of all ages.